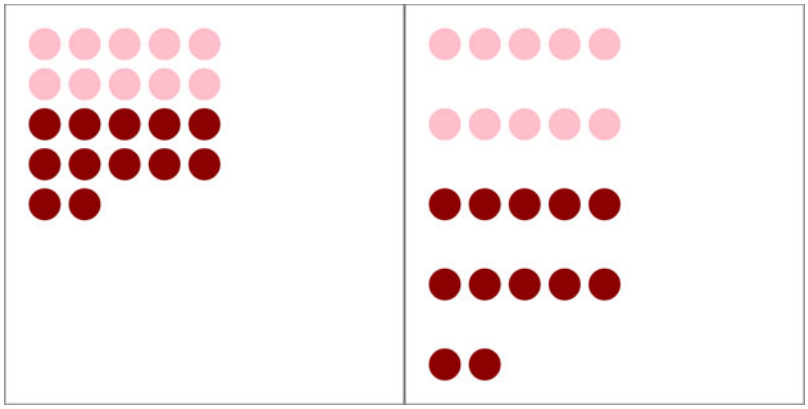


Figure 10.4 The grid layout run in a 200 x 200 size (left) and a 200 x 400 size (right)

Before we move on, it's important that we extend our layout a bit more so that you can better understand how layouts work. In D3 a layout isn't meant to create something as specific as a grid full of circles. Rather *it's supposed to annotate a dataset so you can represent it using different graphical methods*. Let's say we want our layout to also handle squares, which would be a desired feature when dealing with grids.

To handle squares, or more specifically rectangles (because we want them to stretch out if someone uses our layout and sets the height and width to different values), we need the capacity to calculate `height` and `width` values. That's easy to add to our existing layout function.

Listing 10.8 Layout code for calculating height and width of grid cells

```
var gridCellWidth = gridSize[0] / columns;
var gridCellHeight = gridSize[1] / rows;

//other code

  for (var i = 1; i <= rows; i++) {
    for (var j = 1; j <= columns; j++) {
      if (data[cell]) {
        data[cell].x = gridXScale(j);
        data[cell].y = gridYScale(i);
        data[cell].height = gridCellHeight;        <-- New code
        data[cell].width = gridCellWidth;
        cell++;
      }
      else {
        break;
      }
    }
  }
```

And with that in place, we can call our layout and append `<rect>` elements instead of circle elements. We can update our code as in listing 10.9 to offset the `x` and `y`